

Student Lab Setup Guide

Python Programming Course
First-Class Environment Setup Handout

Objective

During your first class session, you will set up a standard Python development environment on your laptop, with the instructor's support. Work through the checklist below in order. By the end of the session, every box should be checked.

- ☐ Python 3.13 installed and added to PATH
- ☐ A virtual environment created with venv
- ☐ Visual Studio Code installed
- ☐ The VS Code Python extension (Microsoft) installed
- ☐ Git installed and configured with your name and email
- ☐ A GitHub account created and verified
- ☐ Windows 10 or 11 (already provided on lab/course laptops)

Note: You do not need Anaconda, Miniconda, Jupyter-with-Anaconda, or any other Python distribution for this course. Installing one alongside the setup below can cause PATH conflicts — please skip it unless the instructor tells you otherwise.

Part 1 — Create Your GitHub Account

GitHub will be used to store and submit your programming work. If you already have a GitHub account, you can use your existing account.

Step 1. Open GitHub

Go to: github.com

Step 2. Create an account

If you do not already have an account:

1. Select Sign up.
2. Use an email address that you can access.
3. Choose your GitHub username carefully.
4. Verify your email address.
5. Remember your GitHub username and password.

Note: Your GitHub username will be used when submitting and sharing course work.

GitHub provides a personal account that can be used to create repositories and collaborate on projects.

Part 2 — Install Python 3.13

Python is the programming language used in this course.

Step 1. Download Python

Use the official Python website: [Python 3.13.15 for Windows](#)

For a normal Windows laptop, download the Windows installer (64-bit). Python.org identifies the 64-bit Windows installer as the recommended Windows installer.

IMPORTANT: Do not use the large yellow **Download Python** button on the python.org home page — it now gives Python 3.14, which this course does not use. On the 3.13.15 page, also skip **Download Python install manager** near the top. Scroll down to the **Files** table and choose **Windows installer (64-bit)** — the file is named `python-3.13.15-amd64.exe`.

Step 2. Run the installer

Open the downloaded installer. On the first installation screen:

IMPORTANT: Select `Add python.exe to PATH`

Then select **Install Now** and allow the installation to complete.

Step 3. Check the installation

Open Command Prompt:

- Press Windows + R
- Type: `cmd`
- Press Enter

Type:

```
python --version
```

You should see a version beginning with:

```
Python 3.13
```

You can also check the Python launcher:

```
py --version
```

You should see:

```
Python 3.13.x
```

If Python does not work

Do not install another version of Python immediately. First close Command Prompt, open a new Command Prompt window, and try again. If it still does not work, contact the course instructor or lab support.

Part 3 — Install Visual Studio Code

Visual Studio Code (VS Code) will be the main editor used for writing and running Python programs.

Step 1. Download VS Code

Go to: [Visual Studio Code — Windows Download](#)

Step 2. Install VS Code

Download the Windows User Installer unless your college specifically instructs you to use another option. The User Setup is recommended by Microsoft for most Windows users because it normally does not require administrator permissions. Run the installer and accept the default options.

Step 3. Start VS Code

After installation, open Visual Studio Code. You should see the VS Code welcome/start screen.

Part 4 — Install the Python Extension in VS Code

Installing Python itself does not automatically give VS Code Python programming support. The Python extension connects VS Code to your Python installation and provides features such as code completion, running programs, debugging, and interpreter selection.

Step 1. Open Extensions

In VS Code, select the Extensions icon on the left side, or press:

```
Ctrl + Shift + X
```

Step 2. Search for Python

In the search box, type:

```
Python
```

Install **Python — Microsoft**. Make sure the publisher is **Microsoft**. Do not install an unrelated extension simply because it has "Python" in its name.

Part 5 — Install Git

Git is the version-control system that will be used to manage your programming work and connect your laptop to GitHub.

Step 1. Download Git

Go to the official Git website: [Git for Windows](#)

Download the current Git for Windows installer (**Git for Windows/x64 Setup**).

Step 2. Install Git

Run the installer. For this course, the default installation options are normally sufficient. If the installer asks you to choose an editor, you may select Visual Studio Code. Complete the installation.

Step 3. Check Git

Open a new Command Prompt and type:

```
git --version
```

You should see a Git version number, for example:

```
git version 2.x.x
```

GitHub's documentation recommends installing Git and then configuring your Git username and email.

Part 6 — Configure Git

Git needs to know your name and email address so that your commits can be associated with you. Open Command Prompt or Git Bash and enter:

```
git config --global user.name "Your Full Name"
```

For example:

```
git config --global user.name "Rahul Sharma"
```

Then enter:

```
git config --global user.email "your-email@example.com"
```

Use the email address associated with your GitHub account.

Tip: To keep your personal email address out of your public commits, use the private address GitHub gives you instead. On GitHub go to **Settings → Emails**, tick **Keep my email addresses private**, and copy the address ending in `@users.noreply.github.com`. Use that address in the command above.

Verify the settings

Enter:

```
git config --global user.name
```

and:

```
git config --global user.email
```

The correct name and email should be displayed.

Note: The Git username configured here is the identity associated with your commits; it is not necessarily the same thing as your GitHub username.

Part 7 — Create Your Course Folder

We will keep all Python course work in one main folder. Using Windows File Explorer, create:

```
Documents
  \PythonCourse
```

Your folder should therefore be approximately:

```
C:\Users\YourName\Documents\PythonCourse
```

Do not use a folder such as C:\Program Files or another system-protected location.

Note: If your Documents folder is synced to OneDrive (File Explorer shows a cloud icon or a path containing OneDrive), create the folder directly in your user folder instead: C:\Users\YourName\PythonCourse. Syncing makes the virtual environment slow or locked, and course work should not go to a personal cloud account.

Part 8 — Open the Course Folder in VS Code

Open VS Code. Select File → Open Folder. Select Documents → PythonCourse. Then select Select Folder.

Your PythonCourse folder should now appear in the Explorer panel on the left.

Part 9 — Create Your Virtual Environment

A virtual environment keeps the packages required by this course separate from other Python projects on your laptop. We will use Python's built-in venv module. A virtual environment is associated with a particular project and helps prevent conflicts between packages used by different projects.

Step 1. Open the VS Code Terminal

In VS Code select Terminal → New Terminal. A terminal will appear at the bottom of the VS Code window.

Step 2. Switch the terminal to Command Prompt

On Windows the VS Code terminal opens as **PowerShell** by default. PowerShell blocks the activation script used in Part 10, so this course uses **Command Prompt** instead:

1. Select the small down-arrow (▼) next to the + at the top-right of the terminal panel.
2. Select **Select Default Profile**, then choose **Command Prompt**.
3. Close the current terminal (bin icon) and open a new one with Terminal → New Terminal.

The new terminal prompt should look like C:\Users\YourName\Documents\PythonCourse> (without PS at the start).

Step 3. Check Python

Enter:

```
python --version
```

Confirm that the version starts with Python 3.13.

Step 4. Create the virtual environment

Enter:

```
python -m venv .venv
```

Wait for the command to finish. A new folder called `.venv` should appear in your VS Code Explorer. Your project now looks approximately like this:

```
PythonCourse  
  \.venv
```

Note: Do not delete or modify files inside `.venv` manually.

Part 10 — Activate the Virtual Environment

In the VS Code terminal, enter:

```
.venv\Scripts\activate
```

After activation, you should see something similar to this at the beginning of the terminal prompt:

```
(.venv)
```

For example:

```
(.venv) C:\Users\YourName\Documents\PythonCourse>
```

This indicates that the virtual environment is active.

Note: If you see “running scripts is disabled on this system”, your terminal is still PowerShell. Go back to Part 9, Step 2 and switch to Command Prompt. (Alternatively, run `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` once in PowerShell, answer Y, and try again.)

Verify the Python version

Enter:

```
python --version
```

It should show:

```
Python 3.13.x
```

Part 11 — Select the Virtual Environment in VS Code

VS Code should use the `.venv` environment when running your programs.

Step 1. Open the Command Palette

Press:

```
Ctrl + Shift + P
```

Step 2. Select the interpreter

Type:

```
Python: Select Interpreter
```

Select Python: Select Interpreter. From the list, select the Python interpreter associated with .venv. It may look similar to:

```
Python 3.13.x ('.venv': venv)
```

The exact wording may vary slightly between VS Code versions. The selected Python environment is shown in the VS Code status bar.

Part 12 — Create Your First Python Program

In the VS Code Explorer:

1. Select New File.
2. Name the file: hello.py

Enter the following:

```
print("Hello, Python!")  
print("My Python environment is working.")
```

Save the file:

```
Ctrl + S
```

Part 13 — Run the Program

There are several ways to run Python in VS Code. For this course, use the Run Python File option initially. Select the ► button near the top-right of the editor. Alternatively, right-click inside the Python file and select Run Python File in Terminal.

You should see:

```
Hello, Python!  
My Python environment is working.
```

If you see this output, Python is working correctly.

Tip: You can also run the program yourself from the terminal (with `.venv` active) by typing `python hello.py`. This is how you will run most lab programs in this course.

Part 14 — Test Package Installation

Python packages will be required later in the course. Before installing packages, make sure that your virtual environment is active. Your terminal should begin with `(.venv)`. We can test the package installation mechanism with:

```
python -m pip --version
```

You should receive information about pip. The important point is that packages for this course should normally be installed inside the `.venv` environment, rather than into the global Python installation. For example, a package would normally be installed using:

```
python -m pip install package-name
```

Note: Do not install additional packages unless instructed by the course instructor or laboratory instructions.

Part 15 — Connect Your Work to GitHub

Your course instructor may provide a GitHub repository for the course. You will normally work with a local copy of the repository on your laptop. The general workflow is:

```
GitHub
-> Clone repository
-> Your laptop
-> Write Python code
-> Commit changes
-> Push changes
-> GitHub
```

GitHub describes cloning as creating a local copy of a repository that remains connected to the GitHub repository.

If your instructor gives you a repository URL

Open the VS Code terminal and use:

```
git clone <repository-url>
```

For example:

```
git clone https://github.com/college/python-course.git
```

The actual URL will be provided by your instructor. Then move into the repository folder:

```
cd repository-name
```

and open it in VS Code:

```
code .
```

Note: The first time you clone a private repository or push, a **Sign in with your browser** window opens. Sign in to GitHub there and select **Authorize**. Your GitHub password typed into the terminal will not work — GitHub no longer accepts it for Git commands.

Part 16 — Important Git Commands

You will learn Git properly during the course. For now, remember these basic commands.

Check the status of your work

```
git status
```

Add your changes

```
git add .
```

Save a snapshot of your changes

```
git commit -m "Completed laboratory exercise"
```

Send your commits to GitHub

```
git push
```

Get the latest changes from GitHub

```
git pull
```

A typical working cycle will therefore be:

```
git pull
-> Write / modify Python code
-> Test your program
-> git status
-> git add .
-> git commit -m "Description of work"
-> git push
```

GitHub's documentation provides further guidance on setting up Git and working with repositories.

Part 17 — Do Not Upload Your Virtual Environment

The `.venv` folder belongs to your laptop. It should not be uploaded to GitHub. Your repository should contain your Python source code and project files, but not the complete virtual environment.

Create a file in your project folder called:

```
.gitignore
```

Add:

```
.venv/
```

This tells Git to ignore the virtual environment. You may later add other files and folders to `.gitignore` as instructed during the course.

Part 18 — Final Verification Checklist

Before attending the first programming laboratory, verify all of the following.

Python

- ☐ Python 3.13 is installed. Run: `python --version`. Expected: Python 3.13.x

VS Code

- ☐ Visual Studio Code is installed.
- ☐ The Microsoft Python extension is installed.
- ☐ The VS Code terminal opens as Command Prompt.
- ☐ The Python: Select Interpreter option shows your `.venv` environment.

Virtual Environment

- ☐ A `.venv` folder exists in your project.
- ☐ The terminal shows `(.venv)` when the environment is activated.

Git

- ☐ Git is installed. Run: `git --version`
- ☐ Git username is configured.
- ☐ Git email is configured.

GitHub

- ☐ You have a GitHub account.
- ☐ Your email address has been verified.
- ☐ You can access the course repository provided by the instructor.

Python Program

- ☐ You can create a `.py` file.
- ☐ You can run a Python program from VS Code.
- ☐ Your program produces the expected output.

Final Expected Setup

At the end of this setup, your development environment should look approximately like this:

```
Your Laptop
|-- Python 3.13
|-- Visual Studio Code
|   |-- Microsoft Python Extension
|-- Git
|-- PythonCourse
|   |-- .venv
|   |-- .gitignore
|   |-- hello.py
|   |-- [course files]
|       |
|       v
|   Git
|       |
|       v
|   GitHub
```

Important Rules for the Course

- Use Python 3.13 as specified for this course.
- Use a virtual environment (.venv) for your course projects.
- Do not install Anaconda unless specifically instructed.
- Do not install packages globally unless specifically instructed.
- Do not upload .venv to GitHub.
- Commit your work regularly.
- Push your work to GitHub regularly so that you have a remote copy of your work.
- Do not share your GitHub password or authentication codes with anyone.
- If you encounter an installation error, record the exact error message and show it to the instructor or lab support rather than repeatedly installing different versions.

Official References

- [Python 3.13.15 — Official Release Page](#)
- [Visual Studio Code — Official Download Page](#)
- [Python in Visual Studio Code — Microsoft Documentation](#)
- [Git for Windows — Official Download Page](#)
- [GitHub — Official Website](#)
- [GitHub Documentation — Set up Git](#)

Guide v1.1 — steps tested on Windows 10 with Python 3.13.15 on 26 Sep 2026; links and current versions checked the same day (VS Code 1.139.1, Git for Windows 2.55.0).